Fixing Mushroom HP Bug in Pygame

https://gemini.google.com/app/f652dd4eaf26473a

User prompt: I'm doing a lab where I fix and extend a Pygame Centipede game. I've attached game.py. Mushrooms are supposed to survive 4 hits (MUSHROOM_HP = 4) but they disappear after 3. Find the bug in hit_mushroom and show the corrected function.

Response: The bug is in the condition used to check if the mushroom should be destroyed.

The Bug

In the current implementation:

```
if self.mushrooms[cell] <= 1:
```

Because mushrooms start with `MUSHROOM_HP = 4`, each hit decreases their HP:

- **Hit 1:** HP = 3
- **Hit 2:** HP = 2
- **Hit 3:** HP = 1 → This triggers `<= 1`, causing the mushroom to be deleted after only **3 hits** instead of 4.

The Corrected Function

To fix this, change the condition to check if the HP drops to `0`:

```
    def hit_mushroom(self, cell):
        self.mushrooms[cell] -= 1
        if self.mushrooms[cell] <= 0:
            del self.mushrooms[cell]
            self.score += 5
```

User prompt: Now implement mushroom_color(hp). In draw it's called as mushroom_color(hp) or default_color. I want a mushroom to flash white briefly when it's hit, and to turn red when it has 1 HP left. Show every function I need to change (hit_mushroom, update, draw) and don't touch anything else.

Response: Here is the implementation for `mushroom_color(hp)` and the updated functions (`hit_mushroom`, `update`, and `draw`) to support the red color at 1 HP and a brief white flash when hit.

1. Implement `mushroom_color(hp)`

Add this top-level function:

```
def mushroom_color(hp):
    """Return an (r, g, b) colour for a mushroom with the given hit points, or None for the default."""
    if hp == 1:
        return (255, 0, 0)  # Red when 1 HP left
    return None
```

2. Updated Functions (`hit_mushroom`, `update`, `draw`)

```
    def hit_mushroom(self, cell):
        self.mushrooms[cell] -= 1
        if not hasattr(self, "mushroom_flashes"):
            self.mushroom_flashes = {}
        self.mushroom_flashes[cell] = 0.15  # Flash white for 0.15 seconds

        if self.mushrooms[cell] <= 0:
            del self.mushrooms[cell]
            self.score += 5
            if cell in self.mushroom_flashes:
                del self.mushroom_flashes[cell]

    def update(self, dt, keys):
        if self.state != "play":
            return

        # Update mushroom flash timers
        if hasattr(self, "mushroom_flashes"):
            for cell in list(self.mushroom_flashes.keys()):
                self.mushroom_flashes[cell] -= dt
                if self.mushroom_flashes[cell] <= 0:
                    del self.mushroom_flashes[cell]

        self.invulnerable = max(0.0, self.invulnerable - dt)
        self.x += (keys[pygame.K_RIGHT] - keys[pygame.K_LEFT]) * PLAYER_SPEED * dt
        self.y += (keys[pygame.K_DOWN] - keys[pygame.K_UP]) * PLAYER_SPEED * dt
        self.x = max(10, min(WIDTH - 10, self.x))
```

```
        self.y = max(ZONE_TOP * CELL + 10, min(ROWS * CELL - 10, self.y))
        if keys[pygame.K_SPACE]:
            self.fire()
        self.update_bullet(dt)
        self.timer += dt
        effective_tick = TICK / (wave_speed_bonus(self.wave) or 1)
        while self.timer >= effective_tick:
            self.timer -= effective_tick
            for chain in self.chains:
                for segment in chain:
                    segment.step(self.mushrooms)
        player_cell = (int(self.y // CELL), int(self.x // CELL))
        if self.invulnerable <= 0 and any((s.row, s.col) == player_cell for c in self.chains for s in c):
            self.lives -= 1
            self.respawn()
            if self.lives <= 0:
                self.state = "lose"
        if not self.chains:
            self.wave += 1
            self.spawn_wave()

    def draw(self, screen):
        screen.fill((8, 8, 16))
        for (row, col), hp in self.mushrooms.items():
            flashing = hasattr(self, "mushroom_flashes") and (row, col) in self.mushroom_flashes
            color = (255, 255, 255) if flashing else (mushroom_color(hp) or (200 - (MUSHROOM_HP - hp) * 40
            center = (col * CELL + CELL // 2, row * CELL + CELL // 2)
            pygame.draw.circle(screen, color, center, CELL // 2 - 1)
            pygame.draw.rect(screen, (230, 230, 200), (center[0] - 3, center[1], 6, CELL // 2 - 1))
        for chain in self.chains:
            for index, segment in enumerate(chain):
                center = (segment.col * CELL + CELL // 2, segment.row * CELL + CELL // 2)
                pygame.draw.circle(screen, (240, 200, 60) if index == 0 else (80, 220, 90), center, CELL /
        if self.bullet:
            pygame.draw.rect(screen, (255, 255, 255), (self.bullet.x - 1, self.bullet.y - 6, 3, 10))
        if self.invulnerable <= 0 or int(self.invulnerable * 10) % 2 == 0:
            pygame.draw.polygon(screen, (80, 200, 255), [(self.x, self.y - 10), (self.x + 10, self.y + 10)
        hud = self.font.render(f"Score {self.score}  Lives {self.lives}  Wave {self.wave}  R = reset", Tru
        screen.blit(hud, (10, ROWS * CELL + 6))
        if self.state == "lose":
            label = self.font.render("GAME OVER - Press R", True, (255, 255, 120))
            screen.blit(label, label.get_rect(center=(WIDTH // 2, HEIGHT // 2)))
```

User prompt: Now implement on_segment_hit(segment, score). It's called from split_chain after a segment is destroyed. Add a short spark particle effect at the segment's position, including the code to update and draw the particles each frame. Keep changes minimal.

Response: Here is the implementation for `on_segment_hit(segment, score)`, along with the updates to `update` and `draw` to handle and render the spark particles using a module-level `PARTICLES` list.

1. Spark Particles & `on_segment_hit`

```
PARTICLES = []


def on_segment_hit(segment, score):
    """Called whenever a centipede segment is shot; add sparkles, sounds, or bonus points here."""[cite: 1
    center_x = segment.col * CELL + CELL // 2
    center_y = segment.row * CELL + CELL // 2
    for _ in range(10):
        PARTICLES.append({
            "x": center_x,
            "y": center_y,
            "dx": random.uniform(-150, 150),
            "dy": random.uniform(-150, 150),
            "life": 0.25
        })
```

2. Updated `update` and `draw` Functions

```
    def update(self, dt, keys):
        global PARTICLES
        for p in PARTICLES[:]:
            p["x"] += p["dx"] * dt
            p["y"] += p["dy"] * dt
            p["life"] -= dt
            if p["life"] <= 0:
                PARTICLES.remove(p)
```

```
        if self.state != "play":
            return
        self.invulnerable = max(0.0, self.invulnerable - dt)
        self.x += (keys[pygame.K_RIGHT] - keys[pygame.K_LEFT]) * PLAYER_SPEED * dt
        self.y += (keys[pygame.K_DOWN] - keys[pygame.K_UP]) * PLAYER_SPEED * dt
        self.x = max(10, min(WIDTH - 10, self.x))
        self.y = max(ZONE_TOP * CELL + 10, min(ROWS * CELL - 10, self.y))
        if keys[pygame.K_SPACE]:
            self.fire()
        self.update_bullet(dt)
        self.timer += dt
        effective_tick = TICK / (wave_speed_bonus(self.wave) or 1)
        while self.timer >= effective_tick:
            self.timer -= effective_tick
            for chain in self.chains:
                for segment in chain:
                    segment.step(self.mushrooms)
        player_cell = (int(self.y // CELL), int(self.x // CELL))
        if self.invulnerable <= 0 and any((s.row, s.col) == player_cell for c in self.chains for s in c):
            self.lives -= 1
            self.respawn()
            if self.lives <= 0:
                self.state = "lose"
        if not self.chains:
            self.wave += 1
            self.spawn_wave()

    def draw(self, screen):
        screen.fill((8, 8, 16))
        for (row, col), hp in self.mushrooms.items():
            color = mushroom_color(hp) or (200 - (MUSHROOM_HP - hp) * 40, 80, 170)
            center = (col * CELL + CELL // 2, row * CELL + CELL // 2)
            pygame.draw.circle(screen, color, center, CELL // 2 - 1)
            pygame.draw.rect(screen, (230, 230, 200), (center[0] - 3, center[1], 6, CELL // 2 - 1))
        for chain in self.chains:
            for index, segment in enumerate(chain):
                center = (segment.col * CELL + CELL // 2, segment.row * CELL + CELL // 2)
                pygame.draw.circle(screen, (240, 200, 60) if index == 0 else (80, 220, 90), center, CELL /
        for p in PARTICLES:
            pygame.draw.circle(screen, (255, 255, 150), (int(p["x"]), int(p["y"])), 2)
        if self.bullet:
            pygame.draw.rect(screen, (255, 255, 255), (self.bullet.x - 1, self.bullet.y - 6, 3, 10))
        if self.invulnerable <= 0 or int(self.invulnerable * 10) % 2 == 0:
            pygame.draw.polygon(screen, (80, 200, 255), [(self.x, self.y - 10), (self.x + 10, self.y + 10)
        hud = self.font.render(f"Score {self.score}  Lives {self.lives}  Wave {self.wave}  R = reset", Tru
        screen.blit(hud, (10, ROWS * CELL + 6))
        if self.state == "lose":
            label = self.font.render("GAME OVER - Press R", True, (255, 255, 120))
            screen.blit(label, label.get_rect(center=(WIDTH // 2, HEIGHT // 2)))
```

---

*User prompt: Finally, implement wave_speed_bonus(wave) so later waves are faster. Use 1 + 0.15 * (wave - 1) and cap it at 3.0 so late waves stay playable. Explain how it's used in update.*

Response: 1. Implement `wave_speed_bonus(wave)`

Add this top-level function to your code:

```
def wave_speed_bonus(wave):
    """Return an extra tick-rate multiplier for centipede segments at the given wave, or None for the defa
    return min(3.0, 1 + 0.15 * (wave - 1))
```

2. How It Is Used in `update`

In the game's `update` method, the speed multiplier scales the movement frequency of the centipede through this line:

```
effective_tick = TICK / (wave_speed_bonus(self.wave) or 1)
```

- **Shorter Tick Interval:** `TICK` represents the base time (in seconds) that must pass between centipede movement steps (`0.09` seconds). Dividing `TICK` by the multiplier returned by `wave_speed_bonus(self.wave)` yields a smaller `effective_tick` for later waves.
- **Faster Movement:** Because the tick interval is smaller, the `while self.timer >= effective_tick:` loop triggers step updates more frequently per second, making the centipede move visibly faster.
- **Playability Cap:** The `min(3.0, ...)` wrapper ensures the multiplier stops scaling upward once it reaches `3.0` (at wave 15), preventing the centipede's speed from becoming unplayable.
